Structure Oriented Self Supervised Learning over Graphs

Zilu Li, Tapan Sahni, Nigel Doering, Charles Ye
University of California San Diego, La Jolla, CA 92093
zil055@ucsd.edu, tsahni@ucsd.edu, juy022@ucsd.edu, nfdoerin@ucsd.edu

1 Introduction

Over the past decade, graph learning algorithms especially graph neural networks (GNNs) have shown superior performance across various domains. For real-world graphs, however, it is strenuous and even prohibitive to annotate labels. Therefore, there is an emerging area in both research and industry communities managing to exploit self-supervised learning (SSL) methods in order to deal with the label scarcity issue. SSL has been successfully deployed in both computer vision (CV) and natural language processing (NLP) domains [1, 2]. Nevertheless, unlike images or texts, graph data is more complicated since it is highly structured, inherently dependent, and in non-Euclidean space. Hence, directly applying successful SSL methods from CV or NLP domains is infeasible and it is non-trivial to develop handcrafted SSL strategies specifically for graphs.

Two mainstream families have occupied the current graph SSL community, which are generative-based [3, 4, 5] and contrastive-based [6, 7, 8]. We argue that there are two major limitations of both families. First, both generative and contrastive ideas are borrowed from CV or NLP domains, which are not specialized for the graph data and may ignore the significant structural information. Second, strategies used in the above approaches like a masking graph, augmenting data, and sampling negatives might be highly handcrafted and usually would introduce new noise and bias.

It is true that some work utilizes auxiliary property-based supervision signals from node, edge, or graph level properties, which can be acquired completely from the graph without external knowledge or information [9, 10]. However, such pretext tasks only focus on low-level information such as node degree, which provides a limited view and lacks of robustness.

Therefore, we would like to introduce advanced geometry and topology structure into this area. We argue that the structure of local neighborhoods plays a significant role in characterizing the property of graph entities (*i.e.* nodes and edges) such that if two nodes (edges) share a similar local structure context, they should be close in the representation space. This argument is pretty useful since structure context is usually positively correlated to label semantics. For example, in the application of anomaly detection, anomalies do not establish connections to other anomalies but may conform to a similar structure context locally (since they have similar tactics or activities to disguise, defraud or steal in a local community) [11]. However, most existing graph SSL methods fail to be aware that two distant nodes (connected via many hops) can also be close in the representation space. To this end, we extract two properties describing the node's local structure context, which are ricci curvature [12] and persistent homology (summarized by persistent diagrams) [13] respectively. It is then desirable to set the structure context summaries as free self-supervision signals for GNNs.

2 Related Work

2.1 Graph Self Supervised Learning

Graph SSL methods can be grouped into three categories, which are generation-based, auxiliary-based, and contrast-based methods [14]. First, generation-based methods focus on reconstructing

an aspect of the original graph after some perturbation was applied and then using the input data as supervision signals. The reconstruction component can be focused either on the graph entity feature or graph structure. Under this setting, the loss function can be generally described as:

$$\arg min_{\phi,\theta}\mathcal{L}_{SSL}\Big(p_\phi(f_\theta(G)), G\Big),$$

where $f_\theta(\cdot)$ and $p_\phi(\cdot)$ are graph encoder and pretext decoder respectively. G denotes the perturbed graph. The objective function for this method is typically defined to measure the distance between the original graph and the reconstructed graph. The origin of this method is traced to the development of the autoencoder [15]. One branch of feature generation-based methods follows masked feature regression as was motivated by image inpainting in CV [16]. Specifically, certain features of nodes or edges in the graph are hidden and the SSL method tries to recover these hidden features given the unhidden features. Similarly, Graph Reconstruction is a feature generation method where certain features of nodes or edges are masked and the neighbors of these masked nodes/edges are used to predict the masked features with a GCN [17] encoder. On the structural generation side, the goal is often to reconstruct an adjacency matrix of the graph or some portion of the graph structure. In GAE [18], a GCN-based encoder generates node embeddings which are then decoded by a combination of inner production and sigmoid activation. GAE has been expounded upon with various methods developing such as VGAE [18], which follows a similar methodology, but uses a different encoder. Lastly, some methods have developed that, instead of trying to reconstruct the entire graph, focus on masking edges and reconstructing just the masked edges. EdgeMask [19], for instance, works under this paradigm.

The auxiliary-based methods concentrated on further exploring the underlying graph structural and attributive information in order to construct diverse pretexts and provide self-supervision signals. There are mainly two different approaches which are the Regression-Based Approach (R-APP) and the Classification-based Approach (C-APP). For R-APP, the idea of this approach primarily lies in extracting rich self-supervision signals by leveraging the inherent graph structure [9] where we consider predicting representative node properties, global structural information, the distance between unlabeled nodes and pre-defined cluster in the graph. The intuition behind this is avoiding over-smoothing while enhancing feature transformation in a local graph structure. For C-APP, it relies on constructing pseudo labels for facilitating model training. The overall method was built on the idea of clustering. Specifically, M3S[20]iteratively trains an encoder and assigns psuedo labels to those unlabeled nodes at each training stage. In addition to feature-based clustering, A structural-aware task based on the inherent topology was introduced by You et al. [7] In order to absorb both advantages from attributive and structural-based clustering, CAGNN [21] assigns cluster indices as a pseudo label and refines the clusters by minimizing the edges between clusters.

The last method is about contrast-based approaches which are based on the idea of mutual information (MI) maximization among two augmented graph instances. A contrastive objective is employed where MI between semantically similar graph instances is maximized while it is minimized among unrelated/negative examples. The loss function is modeled as

$$\arg min_{\phi,\theta}\mathcal{L}_{SSL}\Big(p_\phi(f_\theta(G^{(1)}), f_\theta(G^{(2)}))\Big),$$

where $G^{(1)}$ and $G^{(2)}$ are two differently augmented instances of G. There are primarily three different design dimensions of these types of methods. First is graph augmentation, which aims to generate identity-preserving positive samples of the given graph. Existing work on graph augmentation is divided into three types: attribute-based, topological-based and a combination of both. Attribute-based augmentation has node feature masking [22] and node feature shuffling [23] as its variant. Topological augmentation work on graph adjacency matrix by employing edge modification [24] and graph diffusion [25] techniques. Current graph contrastive methods such as GRACE[26] and MVGRL[27] adopt a hybrid of these strategies. The second perspective on contrast-based methods is Graph Contrastive Learning, where we can classify existing works into same-scale and cross-scale contrastive learning, based on the formulation of the pretext decoder $p_\phi(.)$ Same-scale learning distinguishes instances on the scale while cross-scale distinguishes them on multiple granularities. The third perspective on these methods is to estimate the mutual information based on [28], given the representation of a pair of instances. InfoGraph [29] uses the Jensen-Shannon Estimator to provide a lower bound for MI while InfoNCE [30] provides a novel MI estimation consisting of positive and N negative pairs. Furthermore, the objective functions of BYOL [31] and BGRL [31] are also used as non-bound MI estimators.

2.2 Advanced Graph Structure Enhanced GNNs

Despite that the GNN itself embeds some level of structural information over the graph into graph representations, the structure it learns is usually shallow and inadequate. Existing research work has tried to incorporate advanced structure summaries like graph curvature and persistent homology into GNNs to enhance the structural representation power. Some works directly exploit structure summaries into architecture of GNNs based on approaches such as re-weighting [32], edge attention [33], and refinement over a novel network layer [34]. Others manage to re-wire the graph structure by removing entities (edges) that impair the message passing effectiveness or efficiency [35, 36]. However, previous methods only learn the model in an end-to-end training fashion, *i.e.* supervised learning or semi-supervised learning. To the best of our knowledge, one of them has explored the possibility in a self supervised manner, which will be the concern and novel contribution of our work.

3 Proposed Method

In this section, we shed light on our proposed method to learn advanced structural properties. Our core idea is that the graph embeddings should be consistent with structural contexts so that embeddings contain rich information about the structures besides node features and aggregated features. In order to obtain graph embeddings learned from self supervision signal, we deploy an encoder-decoder architecture. The encoder learns the embedding for each node, and the decoder will predict the pseudo label based on the input of embedding. In some literature, the decoder is also denoted as a projection head.

3.1 Notations and Problem Formulation

We illustrate our method over an attributed graph $G(V, E, X)$, where V is the node set with $|v| = n$, E is the edge set with $|E| = m$, and $X = [x_1, x_2, \cdots, x_n]^T \in \mathbb{R}^{n\times d}$ is the d-dimensional attribute matrix associated with all nodes. Over the graph, a GNN encoder f_θ will be trained and the goal is to learn f-dimensional graph embeddings $f_\theta(G) = [h_1, h_2, \cdots, h_n]^T \in \mathbb{R}^{n\times f}$. For convenience, we also denote the learned representation $h_i = [f_\theta(G)]_i$, where $[\cdot]_i$ is a picking function. The structure context of a node v_i is denoted as s_i. We also denote the free supervision signal as pseudo labels $\hat{Y}$, which is obtained completely from the data itself without external annotations or knowledge. The self-supervised learning objective can be formalized as:

$$\min_{\phi,\theta} \mathcal{L}_{SSL}(\hat{Y}, p_\phi(f_\theta(G)),$$

where p_ϕ is a decoder to predict the pseudo labels $\hat{Y}$. In this work, we explore using structure context $S = \{s_1, s_2, \cdots, s_n\}$ as the pseudo labels to guide the neural network for learning the graph embeddings.

3.2 Structure Context of a Node: Persistent Diagrams

The structure context has been studied in many graph-based data mining tasks which proves to be vital in downstream applications. This work tries to align the node embedding and corresponding structure context. To encode such structure context, we propose to estimate the following likelihood of predicting the structure given an arbitrary node:

$$Pr(s_i|v_i).$$

To learn a representation, we introduce a graph encoder f_θ and transform it into an optimization problem:

$$\max_\theta \sum_{v_i\in V} \log Pr(s_i|[f_\theta(G)]_i).$$

A novelty of our work is introducing a persistent diagram as the structure context summary of a node. The persistent diagram is a representation of persistent homology, a powerful tool to summarize topological features, which is exploited to evaluate and study the structural properties of the graph data from multi-scale views [13]. Due to limited space, we leave the details to the persistent diagram in Appendix A. In a few words, persistent diagram is a 2D set $D = \{(b_\sigma, d_\sigma) \in \mathbb{R}^2\}$, where b_σ is the

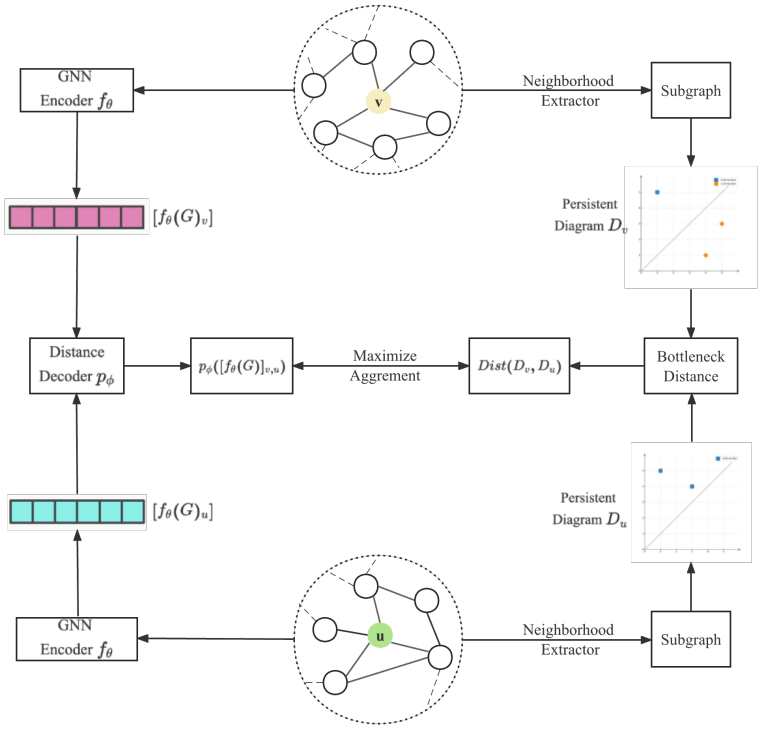


Figure 1: Flowchart of the proposed GraphSSL method. This figure illustrates how to maximize the agreement between two distances given an arbitrary pair of nodes. The middle upper and middle bottom is the snapshot subgraphs of the input graph centered at two nodes v and u respectively. The left flow calculates the distance between two GNN embeddings. The right flow denotes the Bottleneck distance for two persistent diagrams summarized from the k-hop neighborhood of two nodes. The method aims at aligning such two distances by maximizing the agreement between them.

birth time of simplex σ and d_σ is the death time. For each node v_i, a persistent diagram is computed based on the k-hop ego neighborhood network $G_{v_i}^{(k)}$ of v_i. In other words, the neighborhood $G_{v_i}^{(k)}$ is a subgraph of G, consisting of node v_i, its up to k-hop neighbor nodes, and the edges connecting any two included nodes. We denote the computed persistent diagram for $G_{v_i}^{(k)}$ as D_i. Hence the optimization problem can be formulated as:

$$\max_{\theta} \sum_{v_i \in V} \log Pr(D_i | [f_\theta(G)]_i).$$

However, the above objective function is difficult to optimize, inasmuch as the persistent diagram D is a set with variable length. Even though many recent learning approaches have been proposed for set data [37, 38, 39] with respective to perturbation invariance representation, we still argue that those methods are not feasible to determine ground truth pseudo labels in this work.

3.3 A Pair-wise Distance Correspondence Method

In order to deal with the above problem, we present a pairwise-based self-supervised learning strategy. To this end, there is no need to directly predict a variable length set as pseudo labels, but the goal is to maximize the agreement between two distances for any pair of nodes. In the encoder-decoder architecture, the decoder is viewed as the distance between two input embeddings of a pair of nodes. On the other side, the pseudo label is taken as the set distance between the two persistent diagrams of the corresponding pair of nodes. Since the set distance is a pre-calculated scalar, we can directly use deep neural networks to make a prediction without further concerns.

The method starts from sampling pairs of graph entities according to some picking function $[\cdot]_{ij}$ over the learned graph embeddings, where i and j are sampled as pair nodes. We then define distance

functions $Dist(s_i, s_j)$ over the extracted structure properties. As for the choice of distance functions, we utilize the Bottleneck distance for persistent diagrams:

$$Dist(D_i, D_j) = \inf_{\pi \in \Pi} \sup_{x \in Dgm(i)} \|x - \pi(x)\|_\infty,$$

where $\Pi = \pi : D_i \to D_j$ denote the set of all bijections. Please note that in the community of topological and geometrical data analysis, Bottleneck distance is commonly used to measure the similarity between two given persistent diagrams [13]. It is exactly the shortest distance d_s for which there exists a perfect matching, π^*, between points of the two diagrams such that the distance (infinity norm) between any pair of matched points is at most d_s. Computing the Bottleneck distance is an assignment problem in combinatorial optimization and is out of our scope. We refer the readers to the algorithms of bottleneck matching and Bottleneck distance computation in related work [40, 41]. In particular, our implementation to calculate Bottleneck distance follows the work of [41], which gives the most efficient computation to approximate the distance with both theoretical and practical guarantees in literature.

Given the definition of two distances, we summarize our proposed method in Figure 1. It is clear to formulate our loss function as:

$$\min_{\phi,\theta} \mathcal{L}_{SSL}\Big(p_\phi([f_\theta(G)]_{i,j}), Dist(D_i, D_j)\Big),$$

where pretext task decoder p_ϕ can be viewed as distance prediction based on graph embeddings, and SSL is a regression error such as MSE. The loss function above suggests that the learned distance of graph representations of a sampled pair of nodes should be consistent with the distance between their local structures.

3.4 Optimising the sampling function

Our model training relies on sampling a pair of nodes to learn the embeddings by maximizing the agreement between the decoder output and the bottleneck distance. Now since the bottleneck distance quantifies the topological similarity between two nodes, it is easy to see that some node pairs would be more helpful in training than others. Hence, we used a simple linear network with a softmax layer to learn the sampling function for each node. Pairs are sampled according to the probabilities in the softmax output. Our experiments showed a modest jump (around 3% for AUC) in performance with this, though the training time increased. We would like to experiment with this in the future.

4 Experiments

4.1 Datasets

We evaluate our method performance on the Elliptic dataset[1] [42]. Elliptic is a Bitcoin transaction network that aims to detect money laundering activities, where nodes are transactions and edges denote Bitcoin currency (BTC) flow. The dataset maps Bitcoin transactions (*i.e.* nodes) to two categories, the licit (*e.g.* exchanges, wallet providers, miners, licit services) and the illicit (*e.g.* scams, malware, terrorist organizations, ransomware, Ponzi schemes). Statistically, there are 203,769 node transactions and 234,355 directed edge currency flows. The dataset is also highly skewed over the labels, where around 2% nodes are annotated as illicit, while 21% are licit. The labels of the remaining transaction nodes are unknown. Each node, regardless of whether the label is known or not, is associated with 166 feature attributes. The first 94 attributes describe local information for the transaction, *e.g.* time step, transaction fee, output volume. The remaining 72 attributes are called aggregated features, which are obtained by aggregating transaction information one-hop backward or forward from the centre node. We refer the readers for more information about Elliptic dataset to the paper [42].

The second dataset we use for performance evaluation is CORA [43], which is a benchmark dataset commonly used in graph machine learning. Cora graph comprised 2708 scientific publications classified into 7 categories. Each of the 2708 publications represents a node and the 5429 edges between the nodes are the citations between publications. Each node contains 1433 one-hot feature

[1] https://www.kaggle.com/datasets/ellipticco/elliptic-data-set

attributes representing whether the paper includes a specific word in the original corpus. More details of the dataset can be found in the original paper [43]. We use this dataset to evaluate performance for a general application task, which is more balanced over label distribution rather than the Elliptic dataset. Our implementation can be found on Github.[2]

4.2 Evaluation Metrics

Anomaly detection is a binary classification task, but labels of real-world data are highly skewed and imbalanced since anomalies are rare entities. Hence we will evaluate our performance according to three metrics commonly used in imbalanced classification tasks: Area under the ROC Curve (AUC), Geometric Mean (GMean), and macro-F1. Among them, GMean calculates the geometric mean of the true positive rate (TPR) and true negative rate (TNR):

$$GMean = \sqrt{TPR \cdot TNR} = \sqrt{\frac{TP}{TP+TN} \cdot \frac{TN}{TN+FP}}$$

For the CORA dataset, we simply use accuracy as the evaluation metric as the 7 classes is fairly balanced across the dataset.

4.3 Experimental Settings

Comparison Methods. For both tasks, anomaly detection on Elliptic and article classification on CORA, we compare the performance of our method against baseline and state-of-the-art Graph SSL methods. To illustrate, we deploy the models to be compared including GCN[17], GraphSAGE[44], DGI[23], and SSLDegree[9], where GCN and GraphSAGE act as baselines, DGI represent another state of the art SSL approach, and SSLDegree set its self-supervision signal as node degrees, a piece of low-level structural information.

Research Questions. Our experiments are designed to answer the following research questions:

- How does our method perform compared to baseline and SOTA methods on anomaly detection and article classification tasks?
- Which types of graph data does our method work best on?
- Is our method showing robust ability towards different types of graphs and tasks?

We expect that for a task such as anomaly detection, the local structure of a node will be highly correlated with the label of a node as licit or illicit. We assume anomalous node acts differently in terms of edge connectivity pattern from benign ones, which will lead to distinguished structural property of the local neighborhood for an anomalous node. For this task tested on the Elliptic dataset, our proposed method should outperform other methods, including both supervised and self-supervised learning methods. As we reason that the learned embeddings of the node are supervised by the high-level structural information, namely, persistent homology features, which power the embeddings to distinguish between different structural patterns for local neighborhoods.

We also expect that for an article citation graph and the article classification task, such as the CORA dataset, our method may still be competitive among self-supervised methods and even with supervised methods. For such a citation graph, the label distribution (category) of a node may be in-explicitly correlated to its local structure. For example, a highly cited paper exists for every subject, so the SSLDegree method is likely to fail to learn a reasonable representation. However, our method can capture high-level structural information including loops and graph curvature. Such advanced features may help the model be robust to low-level structure perturbation like node degrees. Moreover, considering the fact that the existing of inter-discipline articles, our method is expected to learn such kind of specific citation patterns which others may fail to capture.

Experiment details. To evaluate the performance of the models, we are initially using a standard train/test approach. We split our Elliptic graph data (the nodes with known labels) into a 5% train set and a 95% test set. We then trained supervised GCN and GraphSAGE models to serve as our baselines for anomaly detection performance. For both GCN and GraphSAGE, we use the Adam algorithm for optimization and trained the models for 200 epochs. For DGI, SSLDegree, and GraphSSL, we

[2] https://github.com/tapan0007/GraphSSL

Model	Macro F1	Geometric Mean	AUC
GCN	0.51	0.71	0.73
GraphSAGE	0.43	0.63	0.68
DGI	0.68	0.51	0.63
SSLDegree	0.75	0.82	0.82
GraphSSL	**0.78**	**0.82**	**0.83**

Table 1: Model Performance on Anomaly Detection for the Elliptic Dataset.

Model	Accuracy
GCN	0.80
GraphSAGE	0.86
DGI	0.79
SSLDegree	0.47
GraphSSL	**0.81**

Table 2: Model Accuracy on CORA Dataset.

utilize the entire Elliptic dataset for training the SSL component. For the downstream task, we train a simple multi-layer perceptron for classification on the train split. We follow the same criteria for the CORA dataset. We then compare the models' performance using our anomaly detection metrics on the Elliptic dataset and accuracy for the CORA dataset.

5 Results and Analysis

After testing the models on the anomaly detection task we obtained the results in Table 1. We notice that for all three metrics, the GraphSSL model outperformed any other comparison methods. GraphSSL outperforms the supervised baselines by a large margin, and also a SOTA self-supervised learning method, DGI. GraphSSL also gives a competitive performance with SSLDegree, despite that the latter also outperforms the other three methods. To answer our Research Questions 1, and 2, we find that structural information helps in detecting anomalies. The result suggests that for some downstream tasks such as anomaly detection, where topological and geometrical structure plays a significant role to characterize the node features, our proposed GraphSSL can work much better than other methods.

We answer other research questions by the results for the CORA dataset as shown in Table 2. Our assumption that our proposed GraphSSL should be also competitive in the CORA dataset, is verified by the fact that GraphSSL comes to the second best in terms of accuracy. We also find that SSLDegree has a huge performance gap towards other methods since the degree is low-level structure information and is vulnerable to perturbations. However, GraphSSL utilizes high-level structure information and is therefore more robust. We also reason that GraphSSL is outperformed by GraphSAGE since (1) the structural information is less important in citation graph; (2) the CORA dataset is small, which provides us limited data to learn the node embeddings; (3) GraphSAGE is a supervised method guaranteeing fancy performance for a small dataset.

5.1 Graph Embeddings & Visualization

Given the high-dimensional graph embeddings learned from the self-supervised learning models, we exploit the graph visualization technique to visualize them in lower dimensions. Visualization can help identify patterns and clusters in the graph that might not be easily visible in the original representation. To this end, we utilize t-SNE (t-Distributed Stochastic Neighbor Embedding) [45], a popular technique used for visualizing high-dimensional data in a low-dimensional space. t-SNE visualization can help identify groups of nodes that are similar in terms of their neighbourhood structure. Nodes that are close that each other in the t-SNE plot are likely to have similar embeddings and therefore similar structural properties. In our paper, we think in large networks, labelled data are just a small part of it and sometimes takes many efforts and time for human workers to label

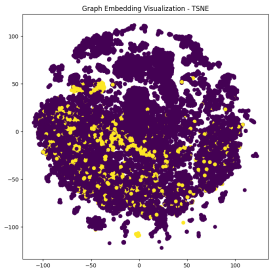


Figure 2: DGI embedding visualization - TSNE

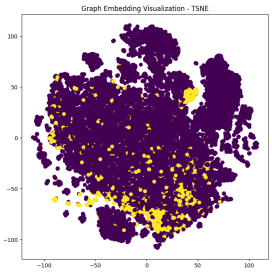


Figure 3: SSL Degree embedding visualization - TSNE

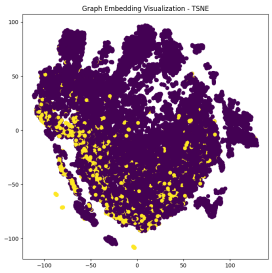


Figure 4: GraphSSL embedding visualization - TSNE

them. So we compared our model with the other two self-supervised graph learning models and look at their embedding visualization. Figure 2 and Figure 3 represent the graph embeddings T-SNE visualization of the DGI model and SSL Degree model respectively, it is hard to tell the clear boundary between the anomaly node in yellow color and the normalcy node in purple color, thus Self-supervised learning methods have to introduce a classifier in order to further classify the labels of the nodes from a disorganized clustering. On the other hand, from Figure 4, we can see the majority of anomaly nodes in yellow color are concentrated in the lower left corner of the cluster, indicating a further improvement in our classification ability compared to the previous two models. With this, the classifier in the next step can achieve good results with fewer iterations of training.

6 Conclusion

Graph self-supervised learning has emerged as a promising approach to learning representations from graphs in the absence of labeled data. With the exponential growth of graph-structured data, there is an increasing need for effective unsupervised learning methods that can learn meaningful representations from graphs. In this context, self-supervised learning techniques have gained popularity due to their ability to leverage unlabeled data to learn representations that capture the underlying structure of the graph. One of the key advantages of graph self-supervised learning methods is their ability to perform well in scenarios where there is only a limited amount of labeled data available. In such scenarios, traditional supervised learning methods like GCN and GraphSAGE often struggle to learn effective representations due to the lack of labeled data. Graph self-supervised learning methods, on the other hand, can leverage the available unlabeled data to learn representations that can be used for downstream tasks. Our research project has demonstrated the superiority of our approach over SSL Degree and DGI in a specific task such as node classification at hand for graph self-supervised learning. Our results show significant improvements in performance, highlighting the potential of our method to outperform existing approaches in related tasks. We summarize that our proposed method reaches SOTA performance when applied into graph tasks where structural information really matters like anomaly detection. We also reveal in practice that our method is robust towards some structural perturbations and hence can be generalized to other domains. In the future, we would like to showcase our method for more generalized applications and tasks. We also intend to improve the training efficiency for the GraphSSL, since our current method to compute the bottleneck distance is under $O(n^{1.5})$, limiting our potential to large-scale graph datasets. Overall, our findings contribute to the field of self-supervised learning, demonstrating the effectiveness of a novel approach and its potential impact on related tasks.

References

[1] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, and Neil Houlsby. An image is worth 16x16 words: Transformers for image recognition at scale, 2020.

[2] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[3] Kaize Ding, Jundong Li, Rohit Bhanushali, and Huan Liu. Deep anomaly detection on attributed networks. In *Proceedings of the 2019 SIAM International Conference on Data Mining*, pages 594–602. SIAM, 2019.

[4] Yuning You, Tianlong Chen, Zhangyang Wang, and Yang Shen. When does self-supervision help graph convolutional networks? In Hal Daumé III and Aarti Singh, editors, *Proceedings of the 37th International Conference on Machine Learning*, volume 119 of *Proceedings of Machine Learning Research*, pages 10871–10880. PMLR, 13–18 Jul 2020.

[5] Dongkwan Kim and Alice Oh. How to find your friendly neighborhood: Graph attention design with self-supervision. In *International Conference on Learning Representations*, 2021.

[6] Yixin Liu, Zhao Li, Shirui Pan, Chen Gong, Chuan Zhou, and George Karypis. Anomaly detection on attributed networks via contrastive self-supervised learning. *IEEE transactions on neural networks and learning systems*, 33(6):2378–2392, 2021.

[7] Yuning You, Tianlong Chen, Yang Shen, and Zhangyang Wang. Graph contrastive learning automated. *arXiv preprint arXiv:2106.07594*, 2021.

[8] Petar Veličković, William Fedus, William L. Hamilton, Pietro Liò, Yoshua Bengio, and R Devon Hjelm. Deep Graph Infomax. In *International Conference on Learning Representations*, 2019.

[9] Wei Jin, Tyler Derr, Haochen Liu, Yiqi Wang, Suhang Wang, Zitao Liu, and Jiliang Tang. Self-supervised learning on graphs: Deep insights and new direction, 2020.

[10] Zhen Peng, Yixiang Dong, Minnan Luo, Xiao-Ming Wu, and Qinghua Zheng. Self-supervised graph representation learning via global context prediction. *arXiv preprint arXiv:2003.01604*, 2020.

[11] Zhiwei Liu, Yingtong Dou, Philip S Yu, Yutong Deng, and Hao Peng. Alleviating the inconsistency problem of applying graph neural network to fraud detection. In *Proceedings of the 43rd international ACM SIGIR conference on research and development in information retrieval*, pages 1569–1572, 2020.

[12] Jürgen Jost and Shiping Liu. Ollivier's ricci curvature, local clustering and curvature-dimension inequalities on graphs. *Discrete & Computational Geometry*, 51(2):300–322, 2014.

[13] Tamal Krishna Dey and Yusu Wang. *Computational topology for data analysis*. Cambridge University Press, 2022.

[14] Yixin Liu, Ming Jin, Shirui Pan, Chuan Zhou, Yu Zheng, Feng Xia, and Philip Yu. Graph self-supervised learning: A survey. *IEEE Transactions on Knowledge and Data Engineering*, 2022.

[15] G.E. Hinton and R. R. Salakhutdinov. Reducing the dimensionality of data with neural networks. *Science*, 2006.

[16] J. Donahue T. Darrell D. Pathak, P. Krahenbuhl and A. A. Efros. Context encoders: Feature learning by inpainting. *CVPR*, page 2536–2544, 2016.

[17] T. N. Kipf and M. Welling. Semi-supervised classification with graph convolutional networks. *ICLR*, page 1–14, 2017.

[18] T. N. Kipf and M. Welling. Variational graph auto-encoders. pages 1–3, 2016.

[19] H. Liu Y. Wang S. Wang Z. Liu W. Jin, T. Derr and J. Tang. Self-supervised learning on graphs: Deep insights and new direction. *WWW Workshop*, 2021.

[20] Z. Zhu K. Sun, Z. Lin. Multi-stage self-supervised learning for graph convolutional networks on graph with few labeled nodes. *AAAI*, 2020.

[21] Feng Yu Shu Wu Yanqiao Zhu, Yichen Xu and Liang Wang. Cagnn: Cluster-aware graph neural networks for unsupervised graph representation learning., 2020.

[22] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. Strategies for pre-training graph neural networks. In *International Conference on Learning Representations*, 2020.

[23] Petar Velickovic, William Fedus, William L. Hamilton, Pietro Lio', Yoshua Bengio, and R. Devon Hjelm. Deep graph infomax. *ArXiv*, abs/1809.10341, 2018.

[24] Ziniu Hu, Yuxiao Dong, Kuansan Wang, Kai-Wei Chang, and Yizhou Sun. Gpt-gnn: Generative pre-training of graph neural networks. In *Proceedings of the 26th ACM SIGKDD Conference on Knowledge Discovery and Data Mining*, 2020.

[25] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. Diffusion improves graph learning. In *Conference on Neural Information Processing Systems (NeurIPS)*, 2019.

[26] Yanqiao Zhu, Yichen Xu, Feng Yu, Qiang Liu, Shu Wu, and Liang Wang. Deep Graph Contrastive Representation Learning. In *ICML Workshop on Graph Representation Learning and Beyond*, 2020.

[27] Kaveh Hassani and Amir Hosein Khasahmadi. Contrastive multi-view representation learning on graphs. In *Proceedings of International Conference on Machine Learning*, pages 3451–3461. 2020.

[28] Michael Tschannen, Josip Djolonga, Paul K. Rubenstein, Sylvain Gelly, and Mario Lucic. On mutual information maximization for representation learning. In *International Conference on Learning Representations*, 2020.

[29] Fan-Yun Sun, Jordan Hoffman, Vikas Verma, and Jian Tang. Infograph: Unsupervised and semi-supervised graph-level representation learning via mutual information maximization. In *International Conference on Learning Representations*, 2019.

[30] Yaochen Xie, Zhao Xu, Zhengyang Wang, and Shuiwang Ji. Self-supervised learning of graph neural networks: A unified review. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 45:2412–2429, 2021.

[31] Jean-Bastien Grill, Florian Strub, Florent Altch'e, Corentin Tallec, Pierre H. Richemond, Elena Buchatskaya, Carl Doersch, Bernardo Ávila Pires, Zhaohan Daniel Guo, Mohammad Gheshlaghi Azar, Bilal Piot, Koray Kavukcuoglu, Rémi Munos, and Michal Valko. Bootstrap your own latent: A new approach to self-supervised learning. *ArXiv*, abs/2006.07733, 2020.

[32] Haifeng Li, Jun Cao, Jiawei Zhu, Yu Liu, Qing Zhu, and Guohua Wu. Curvature graph neural network. *Information Sciences*, 592:50–66, 2022.

[33] Qi Zhao, Ze Ye, Chao Chen, and Yusu Wang. Persistence enhanced graph neural network. In *International Conference on Artificial Intelligence and Statistics*, pages 2896–2906. PMLR, 2020.

[34] Max Horn, Edward De Brouwer, Michael Moor, Yves Moreau, Bastian Rieck, and Karsten Borgwardt. Topological graph neural networks. In *International Conference on Learning Representations (ICLR)*, 2022.

[35] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

[36] Zuoyu Yan, Tengfei Ma, Liangcai Gao, Zhi Tang, and Chao Chen. Link prediction with persistent homology: An interactive view. In *International Conference on Machine Learning*, pages 11659–11669. PMLR, 2021.

[37] Manzil Zaheer, Satwik Kottur, Siamak Ravanbakhsh, Barnabas Poczos, Russ R Salakhutdinov, and Alexander J Smola. Deep sets. *Advances in neural information processing systems*, 30, 2017.

[38] Christoph D Hofer, Roland Kwitt, and Marc Niethammer. Learning representations of persistence barcodes. *J. Mach. Learn. Res.*, 20(126):1–45, 2019.

[39] Keyulu Xu, Weihua Hu, Jure Leskovec, and Stefanie Jegelka. How powerful are graph neural networks? In *International Conference on Learning Representations*, 2019.

[40] Alon Efrat, Alon Itai, and Matthew J Katz. Geometry helps in bottleneck matching and related problems. *Algorithmica*, 31:1–28, 2001.

[41] Michael Kerber, Dmitriy Morozov, and Arnur Nigmetov. Geometry helps to compare persistence diagrams. *ACM J. Exp. Algorithmics*, 22, sep 2017.

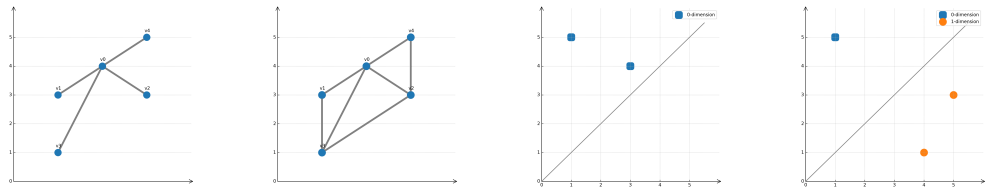
[42] Mark Weber, Giacomo Domeniconi, Jie Chen, Daniel Karl I Weidele, Claudio Bellei, Tom Robinson, and Charles E Leiserson. Anti-money laundering in bitcoin: Experimenting with graph convolutional networks for financial forensics. *arXiv preprint arXiv:1908.02591*, 2019.

[43] Prithviraj Sen, Galileo Namata, Mustafa Bilgic, Lise Getoor, Brian Galligher, and Tina Eliassi-Rad. Collective classification in network data. *AI magazine*, 29(3):93–93, 2008.

[44] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[45] Laurens Van der Maaten and Geoffrey Hinton. Visualizing data using t-sne. *Journal of machine learning research*, 9(11), 2008.

A Persistent Diagrams



(a) (b) (c) (d)

Figure 5: Example of graph descriptor functions and persistent diagrams. (a) and (b): We plot the input graphs with a given descriptor function as the y-axis respectively for two subgraphs. (c) and (d): The persistent diagrams of (a) and (b), computed under extended persistent homology. As a motivating example, it can tell the difference between the two local communities from the perspective of advanced topology features ($i.e.$ persistent diagrams).

This work is interested in obtaining advanced topological features from graph-structured data. Specifically, persistent homology is exploited to evaluate and study the structural properties of the data from multi-scale views. Consider an undirected graph $G = (V, E, X)$, where V is the node set, $E \subseteq V \times V$ is the edge set and $X \in \mathbb{R}^{|V| \times d}$ is the feature matrix. In the graph setting, persistent homology is able to capture 0-dimensional (node-level) features and 1-dimensional (edge-level) features. The simplices K over the graph are defined as nodes and edges.

At first, we obtain a nested sequence of subgraphs such that $\emptyset = G^{(0)} \subseteq G^{(1)} \subseteq \ldots \subseteq G^{(n-1)} \subseteq G^{(n)} = G$. The subgraph sequence induces graph filtrations: $\emptyset = C^{(0)} \subseteq C^{(1)} \subseteq \ldots \subseteq C^{(n-1)} \subseteq C^{(n)} = C$, where each $C^{(i)}$ is defined as a simplicial complex of elements from the respective subgraph. Generally, a sublevel set filtration can be obtained by the descriptor (filtration) function: $C^{(i)} = \{\sigma \in K | f(\sigma) \leq a^{(i)}\}$, where σ is a simplex element of the graph (node or edge) and $a^{(i)}$ is the threshold for descriptor function's value. In this work, we set the descriptor function to be the node degree for any node $f(u) = \delta_u$, and the edge function value is the maximum of two side nodes $f(uv) = \max\{f(u), f(v)\}$. Such filtration provides structural insights by monitoring the evolution of topological pattern occurrences from each $G^{(i)}$ which can be used to calculate their persistence. For 0-dimensional homology, components like σ will be created at time b_σ and killed at time d_σ when the graph is swept through the subgraph filtration. Then the lifespan of the topological feature σ is $d_\sigma - b_\sigma$. We exploit persistent diagrams to record and represent all the topological features: $D = \{(b_\sigma, d_\sigma) \in \mathbb{R}^2\}$.

However, there exists a problem that the whole connected component (0-dimensional feature) and all the cycles (1-dimensional features) will never die according to the above settings. So there would be $(b_\sigma, +\infty)$ points on the diagram and thus cycles are not recorded. To address this limitation, extended persistent homology [**?**] is utilized. Extended persistent homology introduces another super-level set filtration after the sub-level set filtration defined above: $C^{(i)} = \{\sigma \in K | f(\sigma) \geq a^{(i)}\}$. In this case, the descriptor function for an edge is defined as the minimum of two side nodes: $f(uv) = \max\{f(u), f(v)\}$. Since there are two dimensions of topological features, a graph will induce two persistent diagrams. An example of calculating a persistent diagram is illustrated in Figure 5.